

HW5:

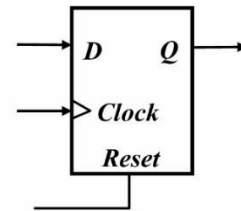
Implement a shift register with left and right shift capability in VHDL, using a structural approach. (Along with a testbench.)

Code:

DFF:

```
ENTITY Reg_ar IS
    PORT ( D, Reset, Clock : IN      ; BIT
           Q                : OUT    BIT );
END Reg_ar ;
```

```
ARCHITECTURE behavioral OF Reg_ar IS
BEGIN
    PROCESS ( Reset, Clock )
    BEGIN
        IF Reset = '1' THEN
            Q <= '0';
        ELSIF rising_edge(Clock) THEN
            Q <= D ;
        END IF ;
    END PROCES
END behavioral ;
```



```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity DFF is
5      port (
6          d : in std_logic;
7          clk : in std_logic;
8          rst : in std_logic;
9          q : out std_logic
10     );
11 end DFF;
12
13 architecture structural of DFF is
14 begin
15     process (clk, rst)
16     begin
17         if rst = '1' then
18             q <= '0';
19         elsif rising_edge(clk) then
20             q <= d;
21         end if;
22     end process;
23 end structural;
```

Shift Register:

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity SR is
5      port (
6          clk : in std_logic;
7          rst : in std_logic;
8          M   : in std_logic;
9          inp : in std_logic;
10         outp : out std_logic_vector(3 downto 0)
11     );
12 end SR;
13
14 architecture structural of SR is
15     component DFF
16     |   port (
17         |       d : in std_logic;
18         |       clk : in std_logic;
19         |       rst : in std_logic;
20         |       q : out std_logic
21         |   );
22     end component;
23
24     signal q_int : std_logic_vector(3 downto 0);
25     signal d_in  : std_logic_vector(3 downto 0);
26 begin
27     d_in(0) <= inp when M='0' else q_int(1);
28     d_in(1) <= q_int(0) when M='0' else q_int(2);
29     d_in(2) <= q_int(1) when M='0' else q_int(3);
30     d_in(3) <= q_int(2) when M='0' else inp;
31
32     DFF0: DFF port map(d => d_in(0), clk => clk, rst => rst, q => q_int(0));
33     DFF1: DFF port map(d => d_in(1), clk => clk, rst => rst, q => q_int(1));
34     DFF2: DFF port map(d => d_in(2), clk => clk, rst => rst, q => q_int(2));
35     DFF3: DFF port map(d => d_in(3), clk => clk, rst => rst, q => q_int(3));
36
37     outp <= q_int;
38 end structural;
```

Testbench:

```
7  architecture behavior of tb_SR is
8      signal clk : std_logic := '0';
9      signal rst : std_logic := '0';
10     signal M : std_logic := '0';
11     signal inp : std_logic := '0';
12     signal outp : std_logic_vector(3 downto 0);
13
14 begin
15     uut: entity work.SR
16         port map (
17             clk => clk,
18             rst => rst,
19             M => M,
20             inp => inp,
21             outp => outp
22         );
23
24     clk_process: process
25     begin
26         while true loop
27             clk <= '0'; wait for 5 ns;
28             clk <= '1'; wait for 5 ns;
29         end loop;
30     end process;
31
32     stim_proc: process
33     begin
34         rst <= '1'; wait for 15 ns;
35         rst <= '0'; wait for 10 ns;
36
37         M <= '1'; inp <= '1'; wait for 50 ns;
38         M <= '0'; inp <= '1'; wait for 50 ns;
39         M <= '1'; inp <= '0'; wait for 50 ns;
40         M <= '0'; inp <= '0'; wait for 50 ns;
41
42         wait;
43     end process;
44 end behavior;
```

Wave:

